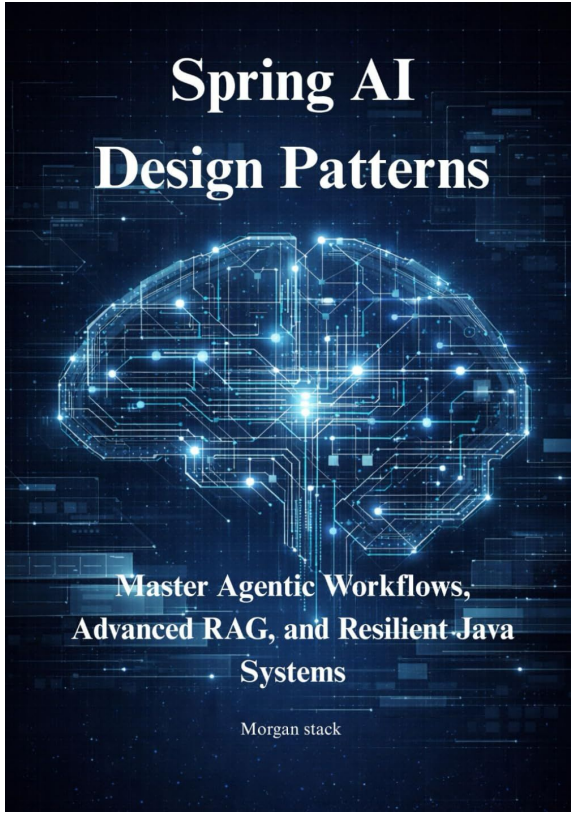
Agenda

- Arch Diagram

- ChatClient
- ChatMemory
- RAG (Retrieval-Augmented Generation)
  - Embedding Model & Vector Store
  - Retrieval (Semantic vs Keyword Search)
  - RAG Importation (Idempotency)
- Tools
- Other Advisors: Logging Observability

- Error Handling (UX improvement)
- Example Design Patterns
  - Routing Pattern
  - Context Window Management

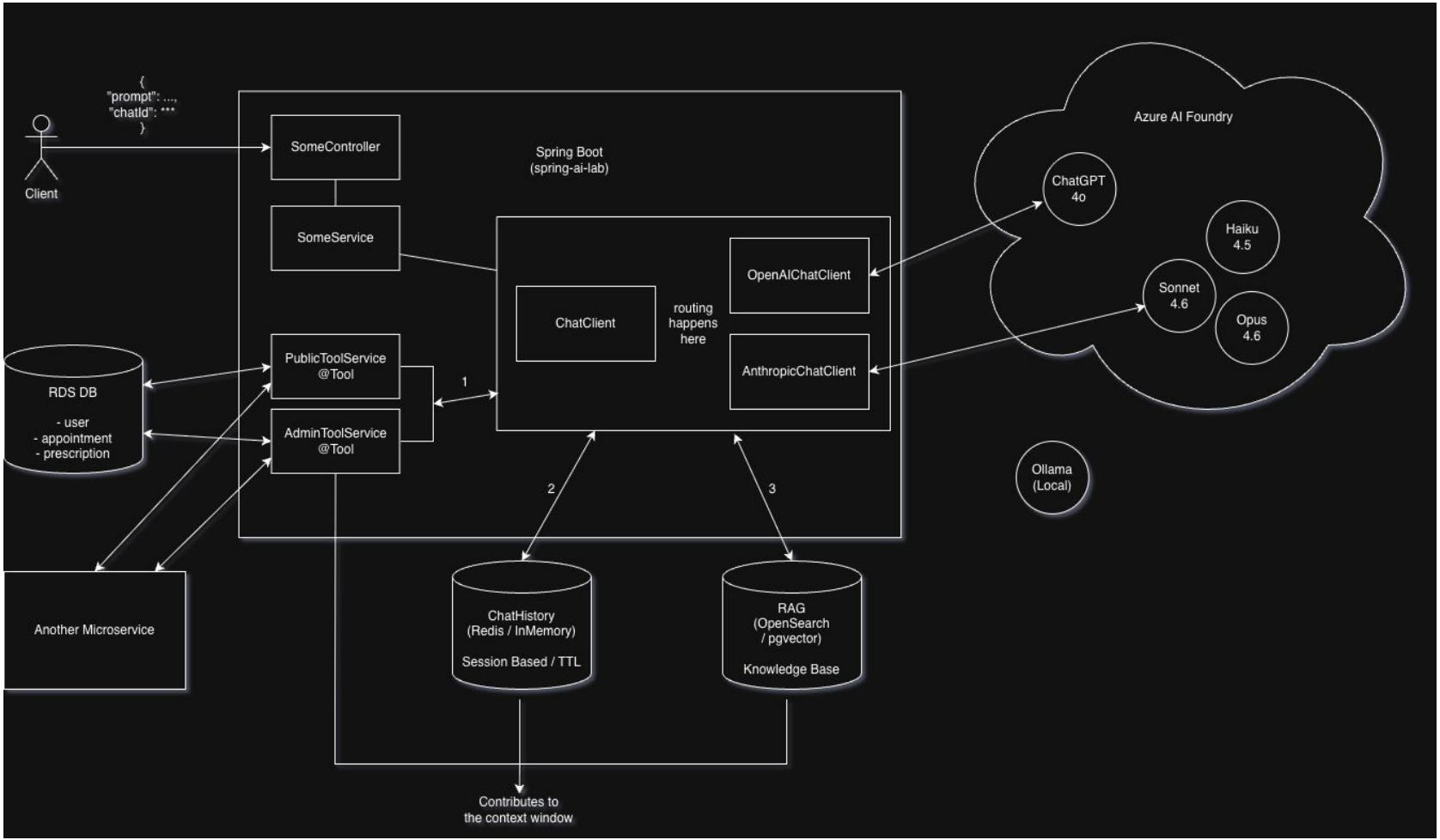
Book Reference



Book reference:
https://www.amazon.de/dp/B0GP6TG8ZJ

Git Repo:
https://github.com/junh-ki/spring-ai-lab
(Spring AI 2)

Arch Diagram



ChatClient (Interface between Java World and AI World)

```java
@Configuration
@RequiredArgsConstructor
public class ChatClientConfig {

    private final PersonaAdvisor personaAdvisor;
    private final PublicBankingToolService publicBankingToolService;

    @Bean
    @Primary
    public ChatClient bankingChatClient(final ChatClient.Builder chatClientBuilder,
                                        final ChatMemory chatMemory,
                                        final VectorStore vectorStore) {
        return chatClientBuilder
            .defaultSystem( text: "You are a helpful banking assistant.")
            .defaultAdvisors( // The Advisors handle the heavy lifting of context management.
                this.personaAdvisor, // persona line to be applied before memory runs
                MessageChatMemoryAdvisor
                    .builder(chatMemory)
                    .build(),
                RetrievalAugmentationAdvisor.builder()
                    .documentRetriever(
                        VectorStoreDocumentRetriever.builder()
                            .vectorStore(vectorStore)
                            .topK(5)
                            .similarityThreshold(0.7)
                            .build() // Define the search logic once
                    )
                    .build(),
                new SimpleLoggerAdvisor()
            )
            .defaultTools(this.publicBankingToolService)
            .build();
    }
```

```java
@Slf4j
@Service
@RequiredArgsConstructor
public class ChatService {

    1 usage
    private static final int DEFAULT_RETRIEVAL_LIMIT = 10;
    private final AiProperties aiProperties;
    private final ChatClient bankingChatClient;

    public String chat(final String userId,
                       final String message) {
        return this.bankingChatClient.prompt() ChatClientRequestSpec
            .user(message)
            .options(
                ChatOptions.builder()
                    .temperature(this.aiProperties.strictTemperature()) // 0.0 (strict) - 1.0 (creative)
                    .build()
            )
            .advisors( AdvisorSpec advisorSpec ->
                advisorSpec
                    .param(ChatMemoryConstant.CONVERSATION_ID, userId) // Pass the conversation ID dynami
                    .param(ChatMemoryConstant.RESPONSE_SIZE, DEFAULT_RETRIEVAL_LIMIT)
            )
            .call() CallResponseSpec
            .content();
    }
```

1. ChatMemory

2. VectorStore for RAG

3. Tools for Actions

4. Other Advisors for I/O Interceptors

a. Temperature for creativity
- the higher the more expressive but indeterministic

b. Session based parameters like conversationId and so on

ChatMemory (Session-based Short-term Memory)

```java
@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, List<Double>> embeddingRedisTemplate(final RedisConnectionFactory redisConnectionFactory) {
        final RedisTemplate<String, List<Double>> redisTemplate = new RedisTemplate<>();
        redisTemplate.setConnectionFactory(redisConnectionFactory);
        redisTemplate.setKeySerializer(new StringRedisSerializer()); // Use String serialization for keys (readable hashes)
        redisTemplate.setValueSerializer(RedisSerializer.json()); // Use JSON serialization for vector data
        return redisTemplate;
    }
}
```

- Technology of choice: Redis
- Session-based Short-term Memory Context with TTL
- Contributes to Context Window

```java
@Configuration
@RequiredArgsConstructor
public class ChatMemoryConfig {

    private final AiProperties aiProperties;

    @Bean
    public ChatMemory chatMemory() {
        return MessageWindowChatMemory.builder()
            .chatMemoryRepository(new InMemoryChatMemoryRepository())
            .maxMessages(this.aiProperties.memoryRetrieveSize())
            .build();
    }

    @Bean
    public ChatMemory redisChatMemory() {
        return MessageWindowChatMemory.builder()
            .chatMemoryRepository(
                new RedisChatMemoryRepository(
                    RedisChatMemoryConfig.builder()
                        .build()
                )
            )
            .maxMessages(this.aiProperties.memoryRetrieveSize())
            .build();
    }
}
```

```java
@Configuration
@RequiredArgsConstructor
public class ChatClientConfig {

    private final PersonaAdvisor personaAdvisor;
    private final PublicBankingToolService publicBankingToolService;

    @Bean
    @Primary
    public ChatClient bankingChatClient(final ChatClient.Builder chatClientBuilder,
                                        final ChatMemory chatMemory,
                                        final VectorStore vectorStore) {
        return chatClientBuilder
            .defaultSystem( text: "You are a helpful banking assistant.")
            .defaultAdvisors( // The Advisors handle the heavy lifting of context management.
                this.personaAdvisor, // persona line to be applied before memory runs
                MessageChatMemoryAdvisor
                    .builder(chatMemory)
                    .build(),
                RetrievalAugmentationAdvisor.builder()
                    .documentRetriever(
                        VectorStoreDocumentRetriever.builder()
                            .vectorStore(vectorStore)
                            .topK(5)
                            .similarityThreshold(0.7)
                            .build() // Define the search logic once
                    )
                    .build(),
                new SimpleLoggerAdvisor()
            )
            .defaultTools(this.publicBankingToolService)
            .build();
    }
```

** A context window is the working memory of an AI model how much information it can remember while generating a response to your prompt.*

RAG (Retrieval-Augmented Generation) - EmbeddingModel & VectorStore

```java
@Configuration
public class VectorStoreConfig {

    @Bean
    @ConditionalOnBean(OpenAiEmbeddingModel.class)
    public VectorStore openAiVectorStore(final OpenAiEmbeddingModel embeddingModel,
                                         final OpenSearchClient openSearchClient) {
        return OpenSearchVectorStore.builder(openSearchClient, embeddingModel)
            .index("spring-ai-docs")
            .dimensions(1536) // depends on the LLM model
            .initializeSchema(true)
            .build();
    }

    @Bean
    @ConditionalOnMissingBean(OpenAiEmbeddingModel.class)
    @ConditionalOnBean(EmbeddingModel.class)
    public VectorStore vectorStore(final EmbeddingModel embeddingModel) {
        return SimpleVectorStore.builder(embeddingModel).build();
    }
}
```

```java
@Bean
public ChatClient bankingChatClient(final ChatClient.Builder chatClientBuilder,
                                    final ChatMemory chatMemory,
                                    final VectorStore vectorStore) {

    return chatClientBuilder
        .defaultSystem( text: "You are a helpful banking assistant.")
        .defaultAdvisors( // The Advisors handle the heavy lifting of context man
            this.personaAdvisor, // persona line to be applied before memory runs
            MessageChatMemoryAdvisor
                .builder(chatMemory)
                .build(),
            RetrievalAugmentationAdvisor.builder()
                .documentRetriever(
                    VectorStoreDocumentRetriever.builder()
                        .vectorStore(vectorStore)
                        .topK(5)
                        .similarityThreshold(0.7)
                        .build() // Define the search logic once
```


```json
"hits": [
  {
    "_index": "my-index",
    "_id": "rgJ5HpIBAG6Nhhq0cY-c",
    "_score": 0.5753642,
    "_source": {
      "id": "park_rocky-mountain",
      "title": "Rocky Mountain",
      "description": "Bisected north to south by the Continental Divide, this portion of the
        Rockies has ecosystems varying from over 150 riparian lakes to montane and subalpine
        forests to treeless alpine tundra."
    }
  }
]
```


- EmbeddingModel
  - Converts text -> numbers

    Input: "Spring AI is cool"
    Output: [0.12, -0.98, 0.44, ...]

  - *Differs depending on the vendor: **OpenAI**, **Bedrock**…*
- VectorStore
  - Stores embeddings (vectors)
  - Semantic Search (Long-term Knowledge Base)
    (a.k.a., similaritySearch)
    - topK - how many hits (results) do I want?
    - similarityThreshold - how good must a match be?
  - Search results (documents) contribute to Context Window
  - Technology of choice:
    OpenSearch, Elasticsearch, pgvector (PostgreSQL Extension)

RAG (Retrieval-Augmented Generation) - Retrieval

```java
@Service
@RequiredArgsConstructor
public class HybridVectorStore implements VectorStore {

    2 usages
    private static final int DEFAULT_TOP_K = 10;
    1 usage
    private static final int DOMINANCE_AVOIDANCE_CONSTANT = 60;
    private final VectorStore vectorStore;
    private final OpenSearchService openSearchService;

    public List<Document> search(final String query) {
        // 1. Run searches in parallel
        final CompletableFuture<List<Document>> vectorFuture = CompletableFuture.supplyAsync(() ->
            this.vectorStore.similaritySearch(
                SearchRequest.builder()
                    .query(query)
                    .topK(DEFAULT_TOP_K)
                    .build()
            )
        );
        final CompletableFuture<List<Document>> keywordFuture = CompletableFuture.supplyAsync(() ->
            this.openSearchService.searchKeywords(
                query,
                DEFAULT_TOP_K
            )
        );
        // 2. Fuse results using RRF (Reciprocal Rank Fusion)
        return fuseResults(
            vectorFuture.join(),
            keywordFuture.join()
        );
    }
```

```java
return chatClientBuilder
    .defaultSystem( text: "You are a helpful banking assi
    .defaultAdvisors( // The Advisors handle the heavy
        this.personaAdvisor, // persona line to be appl
        MessageChatMemoryAdvisor
            .builder(chatMemory)
            .build(),
        RetrievalAugmentationAdvisor.builder()
            .documentRetriever(
                VectorStoreDocumentRetriever.builder()
                    .vectorStore(hybridVectorStore)
```

- Semantic Search:

  “car problems after crash” -> returns documents about “vehicle damage after collision” even if the words “car” or “crash” aren’t used.

- Keyword Search (e.g., BM25):

  “car problems after crash” -> returns documents that explicitly contain words like “car”, “crash”, or “problems”.

- Hybrid Search (RRF):

  Combines semantic and keyword search results by merging ranked lists to improve overall relevance and precision.
  (Reciprocal Rank Fusion)

RAG (Retrieval-Augmented Generation) - RAG Importation

```java
@PostMapping(value = "/parent-child/ingest", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public Map<String, String> ingestParentChild(@RequestPart("file") final MultipartFile multipartFile) {
    if (multipartFile.isEmpty()) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "File must not be empty");
    }
    final String originalFilename = multipartFile.getOriginalFilename();
    final String filename = StringUtils.isNotBlank(originalFilename)
        ? originalFilename
        : "upload.bin";
    this.parentChildIngestor.ingestDocument(toResource(multipartFile, filename));
    return Map.of( k1: "status", v1: "ingested", k2: "filename", filename);
}
```

```java
public void ingestDocument(final Resource resource) {
    final List<Document> rawDocuments = new TikaDocumentReader(resource).get();
    final List<Document> parentChunks = this.parentSplitter.apply(rawDocuments); // 1. Create Parent Chunks (l
    final List<Document> allChildChunks = parentChunks.stream()
        .flatMap( Document parentChunk -> {
            final String parentId = UUID.randomUUID().toString();
            parentChunk.getMetadata()
                .put("doc_id", parentId); // Assign a unique ID to the parent
            this.parentDocStore.put(parentId, parentChunk); // Store the parent safely
            final List<Document> childChunks = this.childSplitter.apply(List.of(parentChunk)); // 2. Create Ch
            childChunks.forEach( Document childChunk -> childChunk.getMetadata().put("parent_id", parentId));
            return this.idempotentMetadataEnricher.apply(childChunks).stream();
        })
        .toList();
    this.vectorStore.add(allChildChunks); // 3. Index ONLY the children in the Vector Store
    log.info(
        "Indexed {} children from {} parents.",
        allChildChunks.size(),
        parentChunks.size()
    );
}
```

```java
@Component
public class IdempotentMetadataEnricher implements DocumentTransformer {

    @Override
    public List<Document> apply(final List<Document> documents) {
        final String now = LocalDateTime.now().toString();
        return documents.stream()
            .map( Document document -> {
                final Map<String, Object> metadata = new HashMap<>(document.getMetadata()); // Create a mutable copy of the metadata
                metadata.putIfAbsent("ingested_at", now); // Add ingestion timestamp
                metadata.putIfAbsent("source", "unknown"); // Ensure a valid source exists
                final String documentContent = document.getText();
                final String uniqueDocContent = documentContent + metadata.get("source_filename"); // If unique field combinations matt
                return StringUtils.isNotBlank(documentContent)
                    ? new Document(
                        HashUtils.computeHash(uniqueDocContent), // Generate ID based on the text content
                        documentContent,
                        metadata
                    ) : null; // Return new document with enriched metadata. Content remains unchanged
            })
            .filter(Objects::nonNull)
            .toList();
    }
}
```

- **Idempotency** is important to avoid duplicating any documents
- Metadata can achieve it
- Target files: pdf or md that contains company or policies or core business wise knowledge
- Will fill up OpenSearch with documents like:

```
{
  "content": "Epikur is founded in 1999",
  "embedding": [0.12, -0.98, 0.44, ...],
  "metadata": {
    "source": "ticket-123"
  }
}
```

Tools

```java
@Slf4j
@Service
public class AdminBankingToolService extends BankingToolService {

    no usages
    @Tool(description = "ADMIN ONLY: Freeze a specific account ID.")
    public String freezeAccount(@ToolParam(description = "The Account ID") final String accountId) {
        log.info("Admin action: Freezing account {}", accountId);
        return "Account " + accountId + " has been frozen.";
    }

    no usages
    @Tool(description = "ADMIN ONLY: Process a refund transaction.")
    public String processRefund(@ToolParam(description = "Transaction ID") final String txId) {
        return "Refund initiated for "+ txId;
    }
}
```

```java
@Service
public class CalendarToolService extends DummyToolService {

    no usages
    @Tool(description = "Schedule a meeting")
    public String scheduleMeeting(@ToolParam(description = "DateTime string") final String dateTime) {
        return "Meeting scheduled for "+ dateTime;
    }
}
```

```java
/**
 * Use the router pattern to determine the intent first, and then load only the relevant tools.
 * (context-aware tool selection)
 */
@GetMapping("/efficient-chat")
public String efficientChat(@RequestParam final String message) {
    final List<DummyToolService> contextToolServices = new ArrayList<>();
    if (message.contains("schedule") || message.contains("meeting")) {
        contextToolServices.add(this.calendarToolService);
    }
    if (message.contains("database")) {
        contextToolServices.add(this.sqlToolService);
    }
    if (CollectionUtils.isEmpty(contextToolServices)) {
        contextToolServices.add(this.generalToolService);
    }
    return this.chatClient.prompt() ChatClientRequestSpec
        .user(message)
        .tools(contextToolServices)
        .call() CallResponseSpec
        .content();
}
```

- *@Tool(description = "...")* Is not meant for humans. It is meant for LLM
- With tools, LLM turns from a simple chatbot into an agent with the actual capabilities beyond chatting
- Could be role-based or prompt keyword-based
- Loaded Tools contribute to Context Window

Other Advisors: Logging Observability

```java
public class ChainLoggerAdvisor implements BaseAdvisor {

    @Override
    public ChatClientRequest before(final ChatClientRequest chatClientRequest,
                                    final AdvisorChain advisorChain) {
        log.info(
            "[{}] >> PROMPT: {}",
            this.stepName,
            Optional.of(chatClientRequest.prompt().getUserMessage()) Optional<Use
                .map(AbstractMessage::getText) Optional<String>
                .orElse(StringUtils.EMPTY)
        );
        log.debug(
            "[{}] >> PARAMS: {}",
            this.stepName,
            chatClientRequest.context()
        );
        return chatClientRequest;
    }

    @Override
    public ChatClientResponse after(final ChatClientResponse chatClientResponse,
                                    final AdvisorChain advisorChain) {
        log.info(
            "[{}] << RESPONSE: {}",
            this.stepName,
            Optional.ofNullable(chatClientResponse.chatResponse()) Optional<ChatRe
                .map(ChatResponse::getResult) Optional<Generation>
                .map(Generation::getOutput) Optional<AssistantMessage>
                .map(AbstractMessage::getText) Optional<String>
                .orElse(StringUtils.EMPTY)
        );
        return chatClientResponse;
    }
}
```

```java
public class EmailChainService {

    no usages
    public String generateResponse(final String userEmail) {
        // --- LINK 1: The Analysis Step ---
        log.info("Step 1: Analyzing email...");
        final String analysis = Optional.ofNullable(
            this.chatClient.prompt() ChatClientRequestSpec
                .advisors(new ChainLoggerAdvisor( stepName: "STEP_1_ANALYZER"))
                .user( PromptUserSpec promptUserSpec ->
                    promptUserSpec
                        .text(ANALYST_PROMPT)
                        .param( k: "raw_email", userEmail)
                )
                .call() CallResponseSpec
                .content()
        ).orElse(StringUtils.EMPTY);
        log.info("--- ANALYSIS RESULT ---");
        log.info(analysis);
        log.info("----------------------");
        // --- LINK 2: The Drafting Step ---
        log.info("Step 2: Drafting response...");
        return this.chatClient.prompt() ChatClientRequestSpec
            .advisors(new ChainLoggerAdvisor( stepName: "STEP_2_WRITER"))
            .user( PromptUserSpec promptUserSpec ->
                promptUserSpec
                    .text(WRITER_PROMPT)
                    .param( k: "analysis_output", analysis)
            )
            .call() CallResponseSpec
            .content();
    }
```

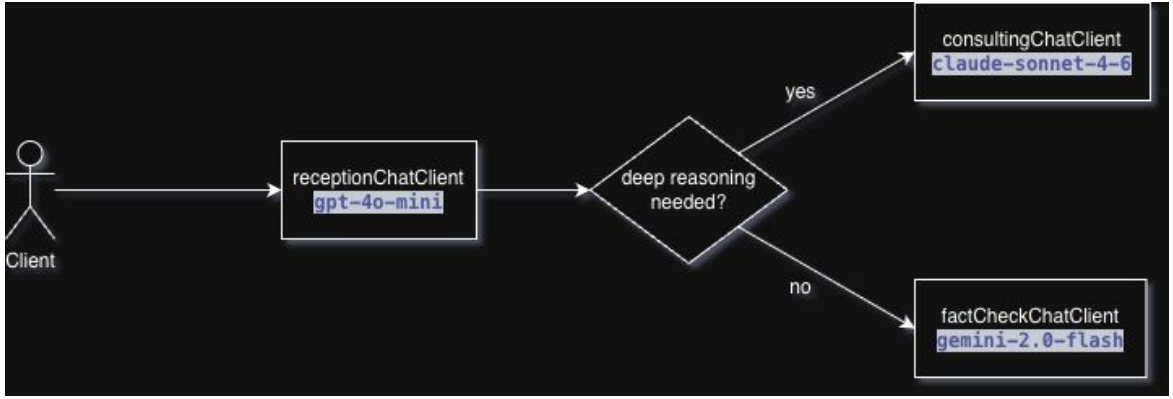
- Prompt-Chaining increases the quality of the response by break down the flow into several steps
- Each step consists of one interaction with LLM
- In case of not very precise result, one can attach ChainLoggerAdvisor
  -> improved observability

Error Handling: UX Improvement

```java
public class RobustWeatherService {
    @Tool(description = "Get the weather forecast for a city.")
    public String getWeather(@ToolParam(description = "City name") final String city) {
        log.info("Fetching weather for: {}", city);
        try {
            return callExternalApi(city);
        } catch (final HttpClientErrorException httpClientErrorException) {
            // Case 1: The AI hallucinated a city or made a typo. We give a hint so the AI can correct itself.
            return String.format(
                "Error: The city '%s' was not found in the database. Please check the spelling or try a nearby major city.",
                city
            );
        } catch (final ResourceAccessException resourceAccessException) {
            // Case 2: Network timeout. We tell the AI this is temporary.
            return "Error: The weather service is currently unreachable (Network Timeout). Please apologize to the user and
        } catch (final Exception exception) {
            // Case 3: Unknown failure. We log the real stack trace for devs, but hide it from the AI.
            log.error("Unexpected tool failure", exception);
            return "Error: An internal system error occurred. Do not retry this specific request.";
        }
    }
```

- Error Handling in the agentic use-case is different from the conventional way
- E.g., if the appointment scheduling service throws NPE
  -> Don’t return the error to the client
  -> LLM should return the feedback in an articulated way

Example Design Patterns - Routing Pattern



gpt-4o-mini (OpenAI)	
	Price
Input	$0.15 / 1M tokens
Output	$0.60 / 1M tokens

claude-sonnet-4-6 (Anthropic)	
	Price
Input	$3.00 / 1M tokens
Output	$15.00 / 1M tokens

gemini-2.0-flash (Google)	
	Price
Input	$0.10 / 1M tokens
Output	$0.40 / 1M tokens

Example Design Patterns - Context Window Management (ChatMemory)

```java
@Bean
public ChatMemory redisChatMemory() {
    return MessageWindowChatMemory.builder()
        .chatMemoryRepository(
            new RedisChatMemoryRepository(
                RedisChatMemoryConfig.builder()
                    .build()
            )
        )
        .maxMessages(this.aiProperties.memoryRetrieveSize())
        .build();
}
```

```java
@Service
@RequiredArgsConstructor
public class ConversationSummarizer {

    1 usage
    private static final String SUMMARY_PROMPT_TEMPLATE = """
        Compress the following conversation history into a concise summary.
        Preserve key facts, user names, and specific constraints.

        HISTORY:
        {history}

        SUMMARY:
        """;
    private final ChatClient chatClient;

    1 usage
    public Message summarize(final List<Message> messages) {
        final String conversationText = messages.stream() Stream<Message>
            .map( Message message -> message.getMessageType() + ": " + message.getText())
            .collect(Collectors.joining( delimiter: "\n"));
        final String summaryText = this.chatClient.prompt() ChatClientRequestSpec
            .user( PromptUserSpec promptUserSpec ->
                promptUserSpec
                    .text(SUMMARY_PROMPT_TEMPLATE)
                    .param( k: "history", conversationText)
            )
            .call() CallResponseSpec
            .content();
        return new SystemMessage(
            textContent: "PREVIOUS CONVERSATION SUMMARY: " + summaryText
        );
    }
}
```

```java
public List<Message> compact(final List<Message> fullHistoricalMessages) {
    if (CollectionUtils.isEmpty(fullHistoricalMessages)
        || fullHistoricalMessages.size() <= MAX_LIMIT) {
        return fullHistoricalMessages;
    }
    // 1. Slice off the oldest chunk (excluding system): Let's say we compress messages from 1 to 10
    final List<Message> messagesToCompress = fullHistoricalMessages.subList(1, 11);
    // 2. Add the previous summary to the context so the AI can merge old + new facts
    messagesToCompress.addFirst(
        new SystemMessage(
            textContent: "Previous Summary: " + this.currentSummary
        )
    );
    // 3. Generate new summary
    final Message newSummaryMsg = this.conversationSummarizer.summarize(messagesToCompress);
    this.currentSummary = newSummaryMsg.getText();
    // 4. Construct the new lightweight history
    final List<Message> newHistoricalMessages = new ArrayList<>();
    newHistoricalMessages.add(fullHistoricalMessages.getFirst()); // Add original Identity
    newHistoricalMessages.add(newSummaryMsg); // Add updated memory
    newHistoricalMessages.addAll(
        fullHistoricalMessages.subList(11, fullHistoricalMessages.size())
    ); // add the remaining recent messages (11 to End)
    return newHistoricalMessages;
}
```

- When max memory retrieve size has been reached
- Do the summary for the previously exchanged messages